

Informe Técnico EP2: Desarrollo de Agente Funcional

Proyecto: Aplicación RAG para evaluación de candidatos por competencias

Asignatura: Ingeniería de Soluciones con IA

Contexto organizacional: Salmones Camanchaca S.A.

Repositorio: <https://github.com/flecaros84/caso>

Alumno: Fabián Lecaros Ampuero

1. Resumen y problema organizacional

El proyecto aborda la revisión documental de postulantes en procesos de selección. En empresas salmoneras, como Salmones Camanchaca S.A., existen cargos técnicos, administrativos y profesionales que requieren comparar varios CV contra un anuncio laboral. Este proceso puede ser lento, repetitivo y difícil de justificar si se realiza solo de manera manual.

La solución desarrollada es una aplicación web que permite seleccionar un anuncio laboral y varios CV. El sistema extrae texto desde archivos, deduce competencias mediante IA generativa, recupera evidencia documental con RAG y genera un ranking con terna recomendada. Para la EP2 se incorporó un flujo alternativo con **agente LangChain**, manteniendo el flujo clásico para no romper la aplicación original.

El agente no reemplaza la decisión humana. Su objetivo es apoyar la preselección documental, dejando evidencia técnica de qué herramientas se usaron, qué decisiones se tomaron y qué memoria se registró durante el análisis.

2. Diseño e implementación del agente

El agente se implementó como una capa de orquestación sobre los servicios ya existentes del backend. No reemplaza el pipeline original, sino que agrega un modo alternativo desde el frontend mediante la opción “Usar agente LangChain”.

Los archivos principales del agente son:

Archivo	Responsabilidad
agent_memory.py	Memoria de corto plazo y persistencia JSON de largo plazo.
agent_planner.py	Plan determinístico y decisiones adaptativas iniciales.
langchain_tools.py	Herramientas LangChain declaradas con StructuredTool.
langchain_recruitment_agent.py	Agente principal con ChatOpenAI, create_openai_tools_agent y AgentExecutor.

La arquitectura permite ejecutar dos caminos:

Frontend



Figura 1. Orquestación general del flujo alternativo con agente LangChain.

3. Herramientas, framework y flujo de trabajo

El framework elegido fue **LangChain** porque permite declarar herramientas, conectar modelos compatibles con OpenAI, ejecutar planificación con agentes y mantener una arquitectura escalable. En este proyecto se usa ChatOpenAI como cliente compatible con API OpenAI, pero apuntando a **GitHub Models** mediante `base_url`. Por eso el modelo seleccionado desde el frontend también afecta al agente.

El agente integra las siguientes herramientas:

Herramienta	Tipo	Función
extract_announcement_text	Consulta	Usa texto manual o extrae el anuncio desde archivo.
extract_cv_text	Consulta	Extrae texto de CV en PDF.
extract_competencies	Razonamiento	Deduce competencias desde el anuncio.
evaluate_candidate_with_rag	Consulta + razonamiento	Recupera evidencia del CV y evalúa competencias.
rank_candidates	Razonamiento / cálculo	Ordena candidatos y genera la terna.
write_analysis_report	Escritura	Genera reportes Markdown y JSON.
save_agent_memory	Memoria	Guarda resumen de sesión en JSON.

El frontend incluye la opción “**Usar agente LangChain**”. Si está desmarcada, se llama al flujo clásico (/api/analyze/start). Si está marcada, se llama al flujo con agente (/api/agent/analyze/start). Esto mantiene compatibilidad y permite comparar ambos modos.

4. Memoria y recuperación de contexto

La memoria se configuró en dos niveles. La memoria de corto plazo vive durante la ejecución y registra pasos, decisiones, llamadas a herramientas, contexto recuperado y resumen final. La memoria de largo plazo se guarda en un archivo JSON local.

Mecanismo	Contenido	Utilidad
Corto plazo	steps, decisions, tool_calls, context_items, final_summary	Permite auditar qué ocurrió en la ejecución actual.
Largo plazo	app/backend/outputs/memory/agent_memory.json	Guarda resumen persistente de la sesión.

Mecanismo	Contenido	Utilidad
Contexto semántico	SimpleRAGIndex + evidencias por competencia	Mantiene continuidad entre CV, competencias y evaluación.

La recuperación de contexto semántico se apoya en el RAG existente. Para cada CV se crea un índice SimpleRAGIndex con fragmentos del documento. Luego, por cada competencia, el sistema recupera evidencia relevante y la entrega al evaluador. Así el LLM no responde desde memoria general, sino desde evidencia concreta del CV.

5. Planificación y decisiones adaptativas

El agente combina un plan determinístico con una planificación generada por LangChain. El plan determinístico asegura estabilidad y define esta secuencia:

1. Preparar anuncio laboral.
2. Extraer competencias.
3. Evaluar candidatos con RAG.
4. Generar ranking.
5. Escribir reporte.
6. Guardar memoria.

Además, AgentExecutor genera una planificación técnica del uso de herramientas, registrada en `agent_trace.planning_output`.

El agente también registra decisiones adaptativas:

Condición	Decisión adaptativa
Texto manual del anuncio disponible	Usar texto manual y omitir OCR.
Sin texto manual	Extraer texto desde archivo con OCR o lectura directa.
Menos de tres CV seleccionados	Generar terna parcial.
Tres o más CV seleccionados	Generar terna completa.
Falla la planificación LLM	Continuar con plan determinístico controlado.

La decisión de mantener ejecución controlada es intencional: el LLM planifica y explica el flujo, pero no ejecuta libremente acciones críticas. Esto reduce errores y mantiene trazabilidad para un prototipo académico.

6. Evidencia de pruebas y resultado esperado

La prueba principal se realizó desde Swagger y luego se habilitó en el frontend. Una respuesta correcta del flujo con agente debe incluir `progress_log` y `agent_trace`.

Ejemplo esperado:

```
{
  "progress_log": [
    "Agente LangChain: preparando anuncio laboral...",
    "Agente LangChain: deduciendo competencias...",
    "Agente LangChain: evaluando CV 1/2..."
  ],
  "agent_trace": {
    "framework": "LangChain",
    "agent_type": "openai_tools_agent",
    "execution_mode": "langchain_planned_controlled_execution"
  }
}
```

Pruebas consideradas:

Prueba	Elemento	Resultado esperado
Flujo clásico	/api/analyze/start	Ranking y reporte sin agent_trace.
Flujo agente	/api/agent/analyze/start	Ranking, reporte, progress_log y agent_trace.
Frontend	Checkbox “Usar agente LangChain”	Permite alternar sin romper el flujo original.
Modelo	Selector desde github_models.json	El modelo seleccionado se usa en análisis y planificación.

7. Justificación técnica, ética y limitaciones

LangChain fue elegido por compatibilidad con agentes, herramientas y clientes tipo OpenAI. FastAPI mantiene endpoints claros y separados para flujo clásico y flujo con agente. RAG se mantiene como mecanismo de recuperación semántica porque permite justificar evaluaciones con fragmentos textuales del CV. La memoria JSON fue seleccionada por simplicidad y transparencia, adecuada para un prototipo beginner.

La aplicación debe usarse solo como apoyo documental. No debe contratar ni descartar automáticamente a una persona. El análisis debe ignorar variables sensibles como edad, género, nacionalidad, fotografía, estado civil, domicilio, familia, religión, salud u opiniones políticas. La recomendación debe basarse en evidencia laboral relacionada con formación, experiencia, conocimientos, certificaciones y funciones realizadas.

Limitaciones actuales:

- los archivos se cargan desde carpetas locales;
- no existe autenticación ni base de datos persistente;
- Tesseract puede fallar con imágenes de baja calidad;
- GitHub Models puede limitar solicitudes;
- algunos modelos del catálogo pueden no estar habilitados para la cuenta;
- el agente mantiene ejecución controlada y no toma decisiones autónomas fuera del flujo definido.

8. Conclusión

El proyecto implementa una solución RAG con IA generativa y agrega un agente funcional como flujo alternativo. El agente integra herramientas de consulta, razonamiento, escritura y memoria; utiliza LangChain como framework; registra planificación, decisiones adaptativas y trazabilidad; y conserva el flujo clásico original.

La solución cumple el requerimiento organizacional porque ayuda a ordenar la revisión inicial de CV y entrega evidencia documental para justificar una terna recomendada. Técnicamente, permite demostrar integración de herramientas, memoria, recuperación semántica, planificación y documentación clara, manteniendo un nivel de complejidad adecuado para un prototipo académico.

Referencias

FastAPI. (s. f.). *FastAPI documentation*. <https://fastapi.tiangolo.com/>

LangChain. (s. f.). *LangChain documentation*. <https://python.langchain.com/>

Microsoft. (s. f.). *GitHub Models documentation*. <https://docs.github.com/>